REWARD crowd

A simple Ethereum testnet DApp for creating and contributing to crowdfunding campaigns. Contributors earn reward ERC-20 tokens, and all transactions are tracked transparently on the blockchain.

Getting Started

Project Purpose

A practical demonstration of decentralized application development using Solidity smart contracts, ERC-20 tokenization, MetaMask wallet integration, and real interaction with an Ethereum test network to illustrate core DApp architecture principles.

Decentralized Crowdfunding Management

Show how smart contracts manage crowdfunding logic and user contributions without centralized control.
Smart Contract Logic

MetaMask-Powered Blockchain Interaction

Demonstrate wallet interaction, token rewards, and blockchain transactions through MetaMask.
Blockchain Interaction

Application Idea

A decentralized crowdfunding platform that enables users to create campaigns, contribute test ETH, and receive internal reward tokens, with all logic governed entirely by smart contracts and no central authority involved.

Campaign Participation

Users can create funding campaigns and support others by contributing test ETH directly through the blockchain.

Tokenized Rewards

Contributors automatically receive educational ERC-20 reward tokens proportional to their participation.

System Architecture

Frontend (JavaScript)

The user interface allows campaign creation, contribution, and data retrieval by sending transaction requests to the blockchain through MetaMask.

MetaMask Wallet

MetaMask acts as a secure bridge between the user and the Ethereum test network, handling account access, transaction signing, and network validation.

Smart Contracts

All crowdfunding rules, contribution tracking, campaign finalization, and token minting are executed on-chain by Solidity smart contracts.

All core business logic is implemented inside smart contracts, ensuring decentralization and transparency.

```
constructor() ERC20("Reward", "RWRD") Ownable(msg.sender) {}

    function setCrowdfundingContract(address _addr) external onlyOwner{
        require(crowdfundingContract == address(0), "Already set");
        crowdfundingContract = _addr;}

    function mint(address to, uint amount) external {
        require(msg.sender == crowdfundingContract, "Not allowed");
        _mint(to, amount);
    }
```

RewardToken Contract

The RewardToken contract implements an educational ERC-20 token that is automatically issued to users as a reward for participating in crowdfunding campaigns, demonstrating core tokenization concepts on the Ethereum test network.

ERC-20 Standard Implementation

The contract follows the ERC-20 standard, providing basic token functionality such as balances, transfers, and supply tracking.

Automatic Minting Mechanism

Tokens are minted automatically when users contribute test ETH to a campaign, linking participation directly to token rewards.

Controlled Minting Access

Only the Crowdfunding smart contract is authorized to mint new tokens, ensuring controlled and secure token issuance.

Crowdfunding Contract

The Crowdfunding contract defines and enforces the full lifecycle of crowdfunding campaigns: creation, contribution tracking, reward distribution, and finalization without any centralized control.

01

Campaign Creation (function createCampaign)

Allows users to create campaigns by specifying a title, funding goal, and duration, establishing transparent rules directly on the blockchain.

02

Contribution Tracking (function contribute)

Records every user contribution in test ETH, updates the total raised amount, and maintains an immutable mapping of individual contributions.

03

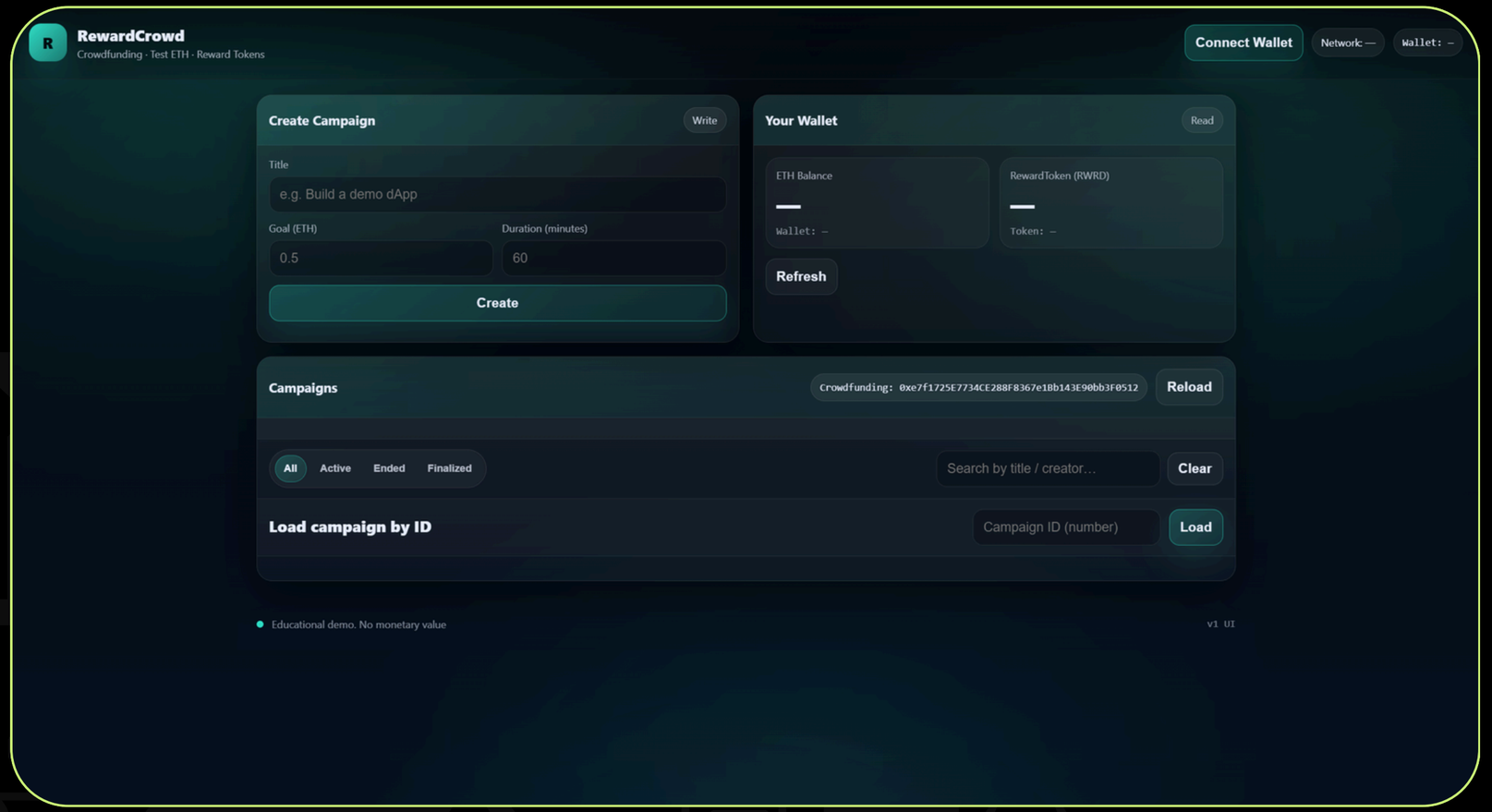
Reward Distribution (rewardToken.mint)

Automatically triggers ERC-20 reward token minting for contributors, linking participation with tokenized incentives.

04

Campaign Finalization (function finalizeCampaign)

Enables campaigns to be finalized either after the deadline or earlier by the creator, securely closing the fundraising process.



The frontend connects through MetaMask and uses ethers.js to interact with the deployed smart contracts.
It loads campaigns and balances using read-only calls (no gas) and performs state-changing actions (create, contribute, finalize) through MetaMask transactions on the local Hardhat network

MetaMask and Frontend Interaction

Smart Contract Deployment & Frontend Logic

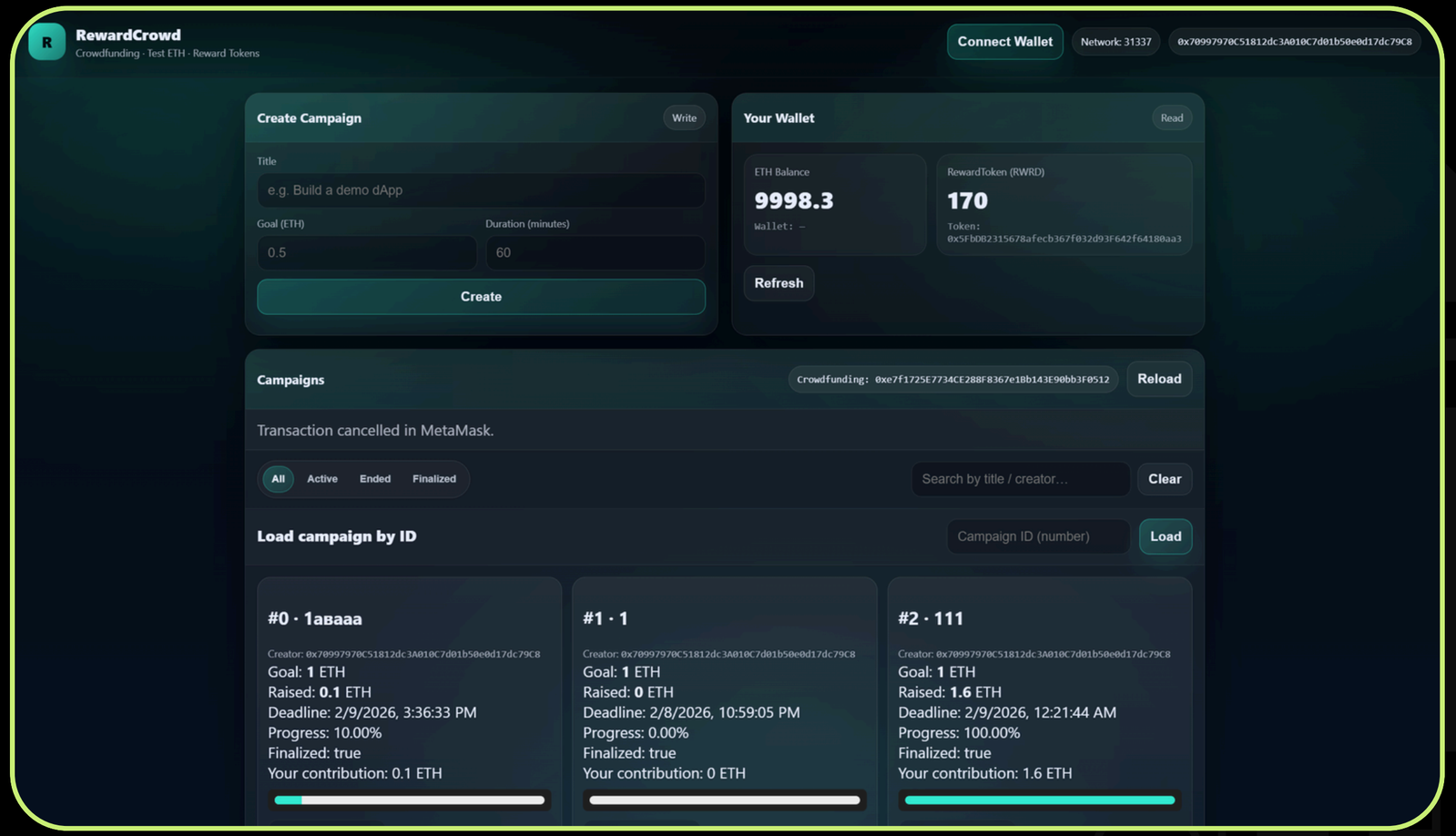
Deploy.js

- Deploys RewardToken and Crowdfunding smart contracts to the local Hardhat blockchain.
- Links contracts by setting the Crowdfunding address inside the token contract.
- Outputs deployed contract addresses required by the frontend.

App.js

- Connects the web UI to MetaMask and the deployed smart contracts via ethers.js.
- Handles core user actions:
- Create campaign
- Contribute ETH
- Finalize campaign
- Load and filter campaigns
- Display ETH and reward token balances
- Updates the interface in real time after blockchain transactions.

Managing Risk and Exit Options



After each successful transaction, the UI reloads campaigns and balances to stay consistent with the latest on-chain state.
The interface also provides basic validation and user-friendly messages for common cases such as ended or finalized campaigns and cancelled MetaMask transactions